

Lesson: If Statements in Python



Introduction

In programming, we often need our code to **make decisions**.

For example:

- If a user enters the correct password → grant access
- If a number is greater than 0 → treat it as positive
- If a student score is high → assign a good grade

This is where **if** statements come in.

An **if** statement allows your program to **execute code only when a condition is true**. This is one of the most important building blocks in Python and is used in nearly every real-world application.

Basic Syntax



```
if condition:  
    # code to run if condition is true
```

Key Rules:

- The condition must evaluate to **True or False**
- A colon `:` is required at the end of the `if` line
- Indentation (spaces) defines the code block

Example 1: Simple If Statement



```
age = 18
if age >= 18:
    print(You are an adult.)
```

How it works:

- `age >= 18` is the condition
- Since it is **True**, the message prints

Comparison Operators

Operator	Meaning
<code>==</code>	Equal to
<code>!=</code>	Not equal to
<code>></code>	Greater than
<code><</code>	Less than

Example:



```
score = 75
if score >= 70:
    print(You passed!)
```

If-Else Statements

Use `else` to handle the opposite condition.

```
if condition:  
    # runs if true  
else:  
    # runs if false
```

Example:

```
temperature = 60  
if temperature > 70:  
    print(It's warm outside.)  
else:  
    print(It's cold outside.)
```

If-Elif-Else (Multiple Conditions)

Use `elif` (short for "else if") to check multiple conditions.



```
if condition1:
    # runs if true
elif condition2:
    # runs if condition1 is false and condition2 is true
else:
    # runs if all conditions are false
```


Example:



```
grade = 85
if grade >= 90:
    print(A)
elif grade >= 80:
    print(B)
elif grade >= 70:
    print(C)
else:
    print(F)
```


Logical Operators

You can combine conditions using:

Operator	Meaning
<code>and</code>	Both conditions must be true
<code>or</code>	At least one condition must be true

Example:



```
age = 25
has_license = True
if age >= 18 and has_license:
    print(You can drive.)
```

Nested If Statements

You can place `if` statements inside other `if` statements.



```
age = 20
if age >= 18:
    print(You are an adult.)
    if age >= 21:
        print(You can legally drink.)
```

Real-World Use Cases

1. User Login Check



```
username = admin
password = 1234
entered_user = input(Enter username: )
entered_pass = input(Enter password: )
if entered_user == username and entered_pass == password:
    print(Access granted)
else:
    print(Access denied)
```

2. Even or Odd Number



```
number = int(input(Enter a number: ))
if number % 2 == 0:
    print(Even number)
else:
    print(Odd number)
```

3. Simple Banking Check



```
balance = 500
withdraw_amount = 200
if withdraw_amount <= balance:
    balance -= withdraw_amount
    print(Withdrawal successful)
else:
    print(Insufficient funds)
```

4. Temperature-Based Decision



```
temperature = int(input(Enter temperature: ))
if temperature >= 80:
    print(Wear shorts)
elif temperature >= 60:
    print(Wear a light jacket)
else:
    print(Wear a coat)
```

Common Beginner Mistakes

Missing indentation

```
if age >= 18:  
print(Adult)    # ERROR
```

Correct:

```
if age >= 18:  
    print(Adult)
```


 Using = instead of ==

```
if age = 18:    # ERROR
```

 Correct:

```
if age == 18:
```


Full Program Example



```
name = input(Enter your name: )
age = int(input(Enter your age: ))
if age < 18:
    print(f{name}, you are a minor.)
elif age < 65:
    print(f{name}, you are an adult.)
else:
    print(f{name}, you are a senior.)
```

Why If Statements Matter

`if` statements allow you to:

- Build interactive programs
- Respond to user input
- Control application logic
- Create real-world systems (apps, games, automation)

Without `if` statements, programs would only run in a straight line with no decision-making.

Summary

In this lesson, you learned:

- `if` statements execute code when a condition is **True**
- `else` handles the opposite case
- `elif` allows multiple conditions
- Logical operators (`and`, `or`, `not`) combine conditions
- Indentation is critical in Python
- `if` statements are used in real-world scenarios like login systems, banking apps, and decision-making logic